In this chapter, you'll see:

- How to specify expected outcomes of your code with rspec-expectations
- What a matcher is
- How to match complex data structures, focusing only on the important details
- How to combine matchers with and/or operators

CHAPTER 10

Exploring RSpec Expectations

In *RSpec Core*, we saw how rspec-core helps you structure your test code into *example groups* and *examples*. But having a sound structure is not enough for writing good tests. If our specs run code without looking at the output, we're not really testing anything—except that the code doesn't crash outright. That's where rspec-expectations comes in. It provides an API for *specifying expected outcomes*.

Each RSpec example should contain one or more *expectations*. These express what you expect to be true at a specific point in your code. If your expectations are not satisfied, RSpec will fail the example with a clear diagnostic message.

In this chapter, we will see how one crucial part of expectations—the matcher—can be combined in useful new ways. Before diving into how expectations work, let's look at a few sample expectations to see what they're capable of:

10-exploring-rspec-expectations/01/expectation_examples.rb

```
ratio = 22 / 7.0
expect(ratio).to be_within(0.1).of(Math::PI)

numbers = [13, 3, 99]
expect(numbers).to all be_odd

alphabet = ('a'..'z').to_a
expect(alphabet).to start_with('a').and end_with('z')
```

Try reading these out loud. "Expect ratio to be within 0.1 of pi." "Expect numbers to all be odd." "Expect alphabet to start with a and end with z." These expectations read exactly like what they verify.

The primary goal of rspec-expectations is *clarity*, both in the examples you write and in the output when something goes wrong. In this chapter, we're going to show you how they work and how to use them in your RSpec examples—or even without RSpec at all!